DESENVOLVIMENTO MOBILE

AULA 8 — HOOKS, ESTADO GLOBAL COM ZUSTAND E PERSISTÊNCIA EM DISCO

useState · useEffect · useContext · Zustand · AsyncStorage · SQLite · CESAR School 2026.2

AGENDA

1. `useState` : ciclo de renderização, atualização funcional e imutabilidade
2. `useEffect` : efeitos colaterais, dependências e limpeza (cleanup)
3. ✋ Checkpoints 1 e 2 · ☕ Intervalo
4. `useContext` : compartilhando contexto sem *prop drilling*
5. `zustand` : estado global simples, ações e seletores
6. **Persistência local em disco**: `AsyncStorage` , middleware `persist` e `expo-sqlite`
7. ✋ Checkpoints 3, 4 e 5 · Atividade aplicada

AO FINAL DA AULA VOCÊ DEVE CONSEGUIR

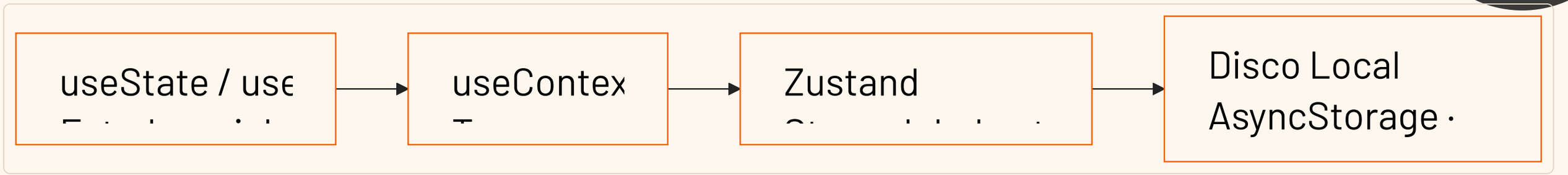
1. Atualizar arrays e objetos no **`useState`** respeitando a imutabilidade
2. Controlar quando um **`useEffect`** roda e limpar timers/listeners no `return`
3. Criar um **Contexto** (`createContext` + `Provider` + custom hook) para tema global
4. Criar uma store global tipada com **`zustand`** e consumir com seletores
5. Persistir dados com **`AsyncStorage`** e o middleware **`persist`** do Zustand
6. Executar tabelas, inserções e consultas locais com **`expo-sqlite`**
7. Decidir entre `useState` , `useContext` , `zustand + persist` e `expo-sqlite`

O PROBLEMA: TELAS ISOLADAS E MEMÓRIA VOLÁTIL

Tela	Arquivo	O que precisa ler/alterar
Home	`(tabs)/index.tsx`	Listar hábitos, adicionar e marcar concluído
Estatísticas	`(tabs)/stats.tsx`	Contar total de hábitos e % concluída
Detalhe	`habit/[id].tsx`	Ver hábito `id`, cronometrar foco e excluir
Configurações	`settings.tsx`	Alternar e lembrar tema claro/escuro do app

> Se cada tela cria seu próprio `useState`, elas não se enxergam — e se ficar só na memória RAM, fechar o app apaga tudo.

DO ESTADO LOCAL AO ARMAZENAMENTO NO DISCO



CADA CAMADA RESOLVE UM ESCOPO E UM TEMPO DE VIDA.

PARTE 1

`useState` E IMUTABILIDADE

AS 2 REGRAS DE OURO DOS HOOKS

- **1. Apenas no nível superior (top level):** nunca chame um Hook dentro de `if`, `for` ou depois de um `return` antecipado.
- **2. Apenas em funções do React:** chame somente dentro de componentes React ou de Custom Hooks (`use...`).

```
// ❌ ERRADO: se id for vazio, o número de hooks chamados muda!
if (!id) return <Text>Carregando...</Text>;
const [seconds, setSeconds] = useState(0);

// ✅ CORRETO: todos os hooks sempre no topo do componente
const [seconds, setSeconds] = useState(0);
if (!id) return <Text>Carregando...</Text>;
```

ATUALIZAÇÃO FUNCIONAL: `prev => ...`

O estado não muda instantaneamente na linha seguinte — o React agenda uma nova renderização. Quando o próximo valor depende do anterior, passe uma **função**:

```
const [streak, setStreak] = useState(0);

// ❌ Lê o mesmo 'streak' congelado desta renderização
const somarDoisErrado = () => {
  setStreak(streak + 1);
  setStreak(streak + 1); // Resultado: +1 em vez de +2
};

// ✅ Recebe o valor mais recente da fila de atualização
const somarDoisCerto = () => {
  setStreak((prev) => prev + 1);
  setStreak((prev) => prev + 1); // Resultado: +2
};
```

IMUTABILIDADE EM ARRAYS E OBJETOS

O React compara a referência antiga com a nova (`novo !== antigo`). Usar `.push()` modifica o mesmo array e a tela **não re-renderiza**.

```
const [habits, setHabits] = useState<Habit[]>([]);

// ✅ Adicionar item: espalhamento (...)
setHabits((prev) => [...prev, novoHabito]);

// ✅ Atualizar item: .map() devolvendo novo objeto
setHabits((prev) =>
  prev.map((h) => (h.id === id ? { ...h, done: !h.done } : h))
);

// ✅ Remover item: .filter()
setHabits((prev) => prev.filter((h) => h.id !== id));
```

✋ CHECKPOINT 1 — `useState`

1. Por que `habits.push(novo); setHabits(habits)` não atualiza a lista na tela?
2. Quando é obrigatório usar `setCount(prev => prev + 1)`?
3. Por que não podemos colocar um `useState` dentro de um `if`?
4. O que acontece com o `useState` de uma tela quando fazemos `router.back()`?

⏱️ 60 segundos em duplas

PARTE 2

`useEffect` E CICLO DE VIDA

SINCRONIZANDO COM O MUNDO EXTERNO

Renderizar deve ser um cálculo puro de `props` e `state`. Quando o componente precisa conversar com algo **fora do React** (timers, sensores, notificações, rede), usamos `useEffect`:

```
useEffect(() => {
  // 1. Efeito: executa DEPOIS que a tela foi desenhada
  console.log('Monitorando hábito:', id);

  // 2. Cleanup (opcional): executa antes de re-executar ou ao desmontar
  return () => {
    console.log('Parando monitoramento de:', id);
  };
}, [id]); // 3. Array de dependências
```

OS 3 MODOS DO ARRAY DE DEPENDÊNCIAS

Sintaxe	Quando o efeito roda	Exemplo prático
`useEffect(fn)` *(sem array)*	Após **toda** renderização	Log de diagnóstico (cuidado!)
`useEffect(fn, [])` *(vazio)*	Só **1 vez** quando a tela monta	Registrar listener de notificação
`useEffect(fn, [id])`	Ao montar + quando `id` mudar	Carregar dados do hábito `id`

> Regra prática: toda variável do componente lida dentro do `useEffect` deve estar no array de dependências.

FECHANDO A TORNEIRA: FUNÇÃO DE CLEANUP

```tsx
export default function FocusTimer() {
  const [seconds, setSeconds] = useState(0);
  const [isRunning, setIsRunning] = useState(true);

  useEffect(() => {
    if (!isRunning) return;

    const timerId = setInterval(() => {
      setSeconds((prev) => prev + 1);
    }, 1000);

    // Roda quando isRunning muda OU quando o usuário sai da tela!
    return () => clearInterval(timerId);
  }, [isRunning]);

  return <Text>Foco: {seconds}s</Text>;
}
```

ARMADILHAS CLÁSSICAS DO `useEffect`

❌ LOOP INFINITO

```
useEffect(() => {
  // Altera a própria dependência!
  setCount(count + 1);
}, [count]);
```

✅ ESTADO DERIVADO SEM EFEITO

```
// Não use useEffect para calcular
// algo a partir de outro estado:
const concluidos = habits.filter(
  (h) => h.done
).length;
```

✋ CHECKPOINT 2 — `useEffect`

1. Qual a diferença entre `useEffect(fn)` e `useEffect(fn, [])`?
2. Em quais dois momentos a função de `cleanup` (`return () => ...`) roda?
3. Preciso exibir quantos hábitos estão concluídos a partir de `habits`. Uso `useEffect`?

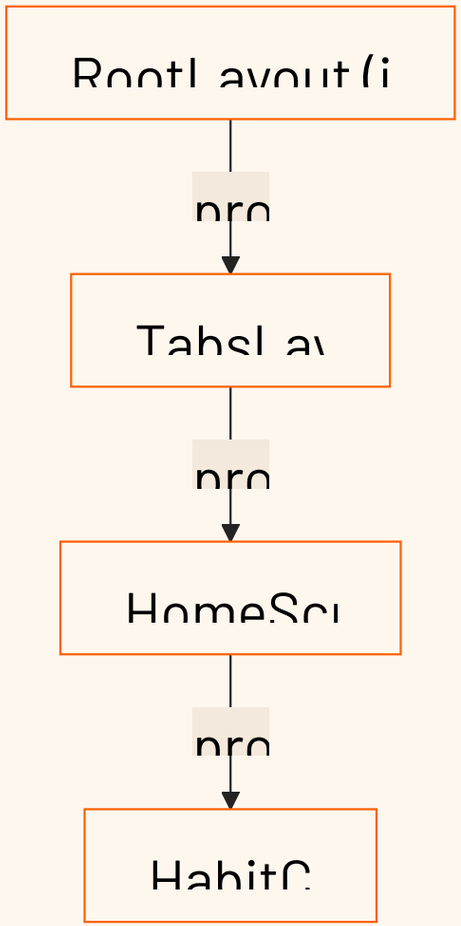
⏱️ 60 segundos em duplas · ☕ Intervalo a seguir

PARTE 3

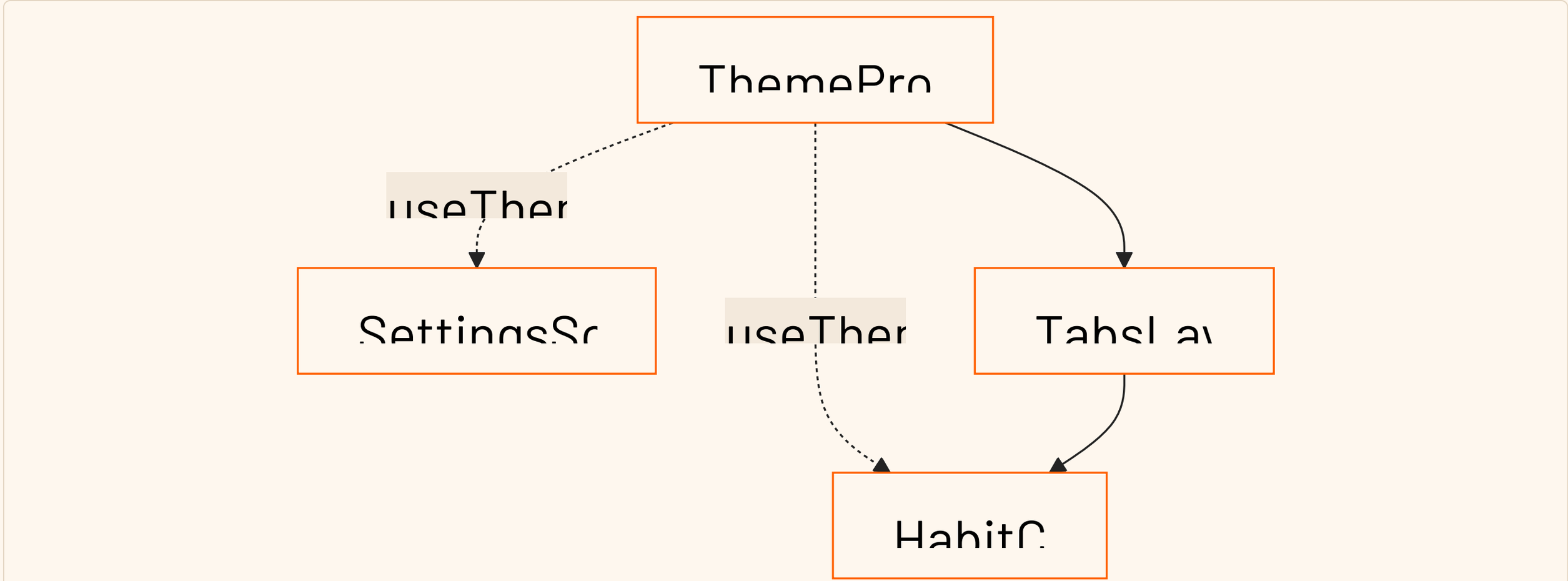
`useContext`

PROP DRILLING × CONTEXTO

COM PROP DRILLING



COM `useContext`



Componentes intermediários não precisam saber que o tema existe.

CRIANDO UM CONTEXTO EM 3 PASSOS

```tsx
// src/context/theme-context.tsx
import { createContext, useContext, useState, type PropsWithChildren } from 'react';

type ThemeContextData = { isDark: boolean; toggleTheme: () => void };

// 1. Criar o contexto
const ThemeContext = createContext<ThemeContextData | undefined>(undefined);

// 2. Criar o Provider
export function ThemeProvider({ children }: PropsWithChildren) {
  const [isDark, setIsDark] = useState(false);
  const toggleTheme = () => setIsDark((prev) => !prev);

  return (
    <ThemeContext.Provider value={{ isDark, toggleTheme }}>
      {children}
    </ThemeContext.Provider>
  );
}

// 3. Criar custom hook para consumo seguro
export function useTheme() {
  const ctx = useContext(ThemeContext);
  if (!ctx) throw new Error('useTheme precisa estar dentro de ThemeProvider');
```

```
  return ctx;
}
```

PLUGANDO O PROVIDER NO `_layout.tsx`

```tsx
// src/app/_layout.tsx
import { Stack } from 'expo-router';
import { ThemeProvider } from '@/context/theme-context';

export default function RootLayout() {
  return (
    <ThemeProvider>
      <Stack />
    </ThemeProvider>
  );
}

// Em qualquer tela (ex: src/app/settings.tsx):
export default function SettingsScreen() {
  const { isDark, toggleTheme } = useTheme();
  return <Switch value={isDark} onValueChange={toggleTheme} />;
}
```

QUANDO USAR (E QUANDO EVITAR) `useContext`

- ✅ **Excelente para:** dados que mudam pouco e afetam muitas telas (Tema claro/escuro, Idioma, Usuário logado).
- ⚠️ **Cuidado com:** dados que mudam com alta frequência ou objetos gigantes.

> Quando o `value` do Provider muda, **todos os componentes que chamam `useContext` re-renderizam** — não há seleção parcial de propriedades.

✋ CHECKPOINT 3 — `useContext`

1. Quais são os 3 passos para criar e expor um Contexto com TypeScript?
2. Onde colocamos o `<ThemeProvider>` em um projeto com Expo Router?
3. Por que evitamos colocar toda a lógica de negócio do app num único Contexto?

⏱️ 45 segundos em duplas

PARTE 4

ESTADO GLOBAL COM `zustand`

POR QUE ZUSTAND?

Zustand é uma biblioteca leve de gerenciamento de estado global para React e React Native:

- 🪶 **Sem `<Provider>` :** a store existe fora da árvore e é acessada via hook
- 🎯 **Seletores cirúrgicos:** só re-renderiza o componente se a fatia selecionada mudar
- 📦 **Estado + Ações juntos:** sem reducers complexos nem actions espalhadas

```
npm install zustand
```

CRIANDO A STORE COM `create`

```ts
// src/store/use-habit-store.ts
import { create } from 'zustand';

export type Habit = { id: string; title: string; done: boolean };

type HabitStore = {
  habits: Habit[];
  addHabit: (title: string) => void;
  toggleHabit: (id: string) => void;
  removeHabit: (id: string) => void;
};

export const useHabitStore = create<HabitStore>((set) => ({
  habits: [
    { id: '1', title: 'Beber 2L de água', done: false },
    { id: '2', title: 'Ler 15 páginas', done: true },
  ],
  addHabit: (title) =>
    set((state) => ({
      habits: [...state.habits, { id: Date.now().toString(), title, done: false }],
    })),
  toggleHabit: (id) =>
    set((state) => ({
      habits: state.habits.map((h) => (h.id === id ? { ...h, done: !h.done } : h)),
    })),
  removeHabit: (id) =>
```

```
    set((state) => ({
      habits: state.habits.filter((h) => h.id !== id),
    })),
}));
```

CONSUMINDO A STORE COM SELETORES

```tsx
// src/app/(tabs)/index.tsx
import { useHabitStore } from '@/store/use-habit-store';

export default function HomeScreen() {
  // ✅ Seletor: assina apenas a propriedade necessária
  const habits = useHabitStore((state) => state.habits);
  const toggleHabit = useHabitStore((state) => state.toggleHabit);

  return (
    <FlatList
      data={habits}
      keyExtractor={(item) => item.id}
      renderItem={({ item }) => (
        <Pressable onPress={() => toggleHabit(item.id)}>
          <Text>{item.done ? '✅' : '⬜'} {item.title}</Text>
        </Pressable>
      )}
    />
  );
}
```

SINCRONIZANDO ABA STATS E ROTA DINÂMICA `[id]`

```tsx
// src/app/(tabs)/stats.tsx
export default function StatsScreen() {
  const habits = useHabitStore((state) => state.habits);
  const done = habits.filter((h) => h.done).length;
  return <Text>Progresso: {done} / {habits.length} concluídos</Text>;
}

// src/app/habit/[id].tsx
export default function HabitDetailScreen() {
  const { id } = useLocalSearchParams<{ id: string }>();
  const habit = useHabitStore((state) => state.habits.find((h) => h.id === id));
  const toggleHabit = useHabitStore((state) => state.toggleHabit);

  if (!habit) return <Text>Hábito não encontrado</Text>;
  return (
    <Pressable onPress={() => toggleHabit(habit.id)}>
      <Text>{habit.title}: {habit.done ? 'Concluído ✅' : 'Pendente ⬜'}</Text>
    </Pressable>
  );
}
```

COMPARATIVO: QUAL ESCOLHER?

Critério	`useState`	`useContext`	`zustand`
Escopo	1 componente	Árvore abaixo do Provider	Global no app
Precisa Provider?	Não	Sim	**Não**
Re-render	Componente atual	Todos os consumidores	Só quem selecionou a fatia
Uso ideal no app	Input de texto, modal	Tema claro/escuro, Auth	Hábitos, filtros, estatísticas

✋ CHECKPOINT 4 — `zustand`

1. Precisamos envolver o `_layout.tsx` com algum Provider para usar o Zustand?
2. Por que usamos `useHabitStore((s) => s.habits)` em vez de `useHabitStore()` vazio?
3. Marquei um hábito como concluído em `/habit/1`. O que acontece na aba Home e na aba Stats?
4. Se eu fechar o app agora e abrir de novo, os hábitos adicionados continuam lá?

⏱️ 45 segundos em duplas

PARTE 5

PERSISTÊNCIA LOCAL EM DISCO

MEMÓRIA RAM × DISCO DO APARELHO

Até agora, `useState`, `useContext` e `zustand` vivem apenas na **memória RAM**. Quando o usuário fecha o app, tudo volta para a estaca zero.

- 🔑 **`AsyncStorage`:** dicionário chave-valor assíncrono em JSON (preferências, flags, listas leves)
- 🔄 **Middleware `persist` do Zustand:** sincroniza a store do Zustand com o `AsyncStorage` automaticamente
- 🗄️ **`expo-sqlite`:** banco de dados SQL relacional completo para grandes volumes e filtros complexos

`AsyncStorage` : CHAVE E VALOR ASSÍNCRONO

```
npx expo install @react-native-async-storage/async-storage
```

Guarda apenas `string` — use `JSON.stringify` ao salvar e `JSON.parse` ao ler:

```
import AsyncStorage from '@react-native-async-storage/async-storage';

// Salvar preferência no disco
export async function salvarTema(isDark: boolean) {
  await AsyncStorage.setItem('@habits:theme', JSON.stringify(isDark));
}

// Ler preferência ao abrir o app (ex: num useEffect com [])
export async function carregarTema(): Promise<boolean> {
  const valor = await AsyncStorage.getItem('@habits:theme');
  return valor !== null ? JSON.parse(valor) : false;
}
```

MIDDLEWARE `persist` DO ZUSTAND

Em vez de chamar `AsyncStorage` manualmente em cada ação, o middleware `persist` grava e recarrega (hidrata) a store sozinho:

```ts
// src/store/use-habit-store.ts
import AsyncStorage from '@react-native-async-storage/async-storage';
import { create } from 'zustand';
import { createJSONStorage, persist } from 'zustand/middleware';

export const useHabitStore = create<HabitStore>()(
  persist(
    (set) => ({
      habits: [],
      addHabit: (title) =>
        set((s) => ({ habits: [...s.habits, { id: Date.now().toString(), title, done: false }] })),
      toggleHabit: (id) =>
        set((s) => ({ habits: s.habits.map((h) => (h.id === id ? { ...h, done: !h.done } : h)) })),
      removeHabit: (id) =>
        set((s) => ({ habits: s.habits.filter((h) => h.id !== id) })),
    }),
    {
      name: '@habits:store', // Chave no AsyncStorage
      storage: createJSONStorage(() => AsyncStorage),
    }
  )
);
```

Atenção ao currying do TypeScript: `create<HabitStore>()(persist(...))`.

`expo-sqlite` : BANCO RELACIONAL NO CELULAR

```
npx expo install expo-sqlite
```

```
import * as SQLite from 'expo-sqlite';

const db = await SQLite.openDatabaseAsync('habits.db');

// 1. Criar tabela (execAsync)
await db.execAsync(`
  CREATE TABLE IF NOT EXISTS habits (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    title TEXT NOT NULL,
    done INTEGER DEFAULT 0
  );
`);

// 2. Inserir ou atualizar com placeholders '?' (runAsync)
await db.runAsync('INSERT INTO habits (title, done) VALUES (?, ?)', 'Beber água', 0);

// 3. Consultar linhas tipadas (getAllAsync)
const pendentes = await db.getAllAsync<{ id: number; title: string; done: number }>(
  'SELECT * FROM habits WHERE done = ? ORDER BY id DESC',
```

```
  0
);
```

`AsyncStorage` + `persist` × `expo-sqlite`

Critério	AsyncStorage	Zustand persist	expo-sqlite
Formato	Chave-valor (`string`)	Store JSON no AsyncStorage	Tabelas SQL relacionais
Sincronização	Manual (`getItem` / `setItem`)	Automática a cada `set()`	Queries SQL (`runAsync`, `getAllAsync`)
Volume ideal	Flags e preferências	Dezenas a centenas de itens	Milhares de registros e histórico
Consultas	Chave exata	Em memória (`.filter()`)	SQL (`WHERE`, `JOIN`, índices)

✋ CHECKPOINT 5 — PERSISTÊNCIA EM DISCO

1. Por que precisamos usar `JSON.stringify` e `JSON.parse` com o `AsyncStorage`?
2. Qual o papel de `createJSONStorage(() => AsyncStorage)` no middleware `persist`?
3. Quando vale a pena usar `expo-sqlite` em vez de salvar um array no Zustand com `persist`?
4. Qual a diferença entre `db.runAsync` e `db.getAllAsync` no `expo-sqlite`?

⏱️ 45 segundos em duplas

TUDO JUNTO NO PROJETO DE HÁBITOS

PRÓXIMOS PASSOS (NÃO É AULA)

- Consumo de APIs remotas e sincronização com servidor (`fetch` , TanStack Query)
- Arquitetura offline-first (sincronizando banco local com nuvem)
- Hooks avançados de otimização (`useMemo` , `useCallback` , `useRef`)

SUAS TELAS CONVERSAM E OS DADOS PERMANECEM

USESTATE · USEEFFECT · USECONTEXT · ZUSTAND · PERSISTÊNCIA LOCAL

Aula 8 — Desenvolvimento Mobile · CESAR School 2026.2

Speaker notes